

PRACTICAL SESSION-8

Task 1 : Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
krishnasree08@krishnasree08: ~$ cd practical8
krishnasree08@krishnasree08: ~/practical8$ pwd
/home/vennela_07/practical8
krishnasree08@krishnasree08: ~/practical8$ nano memory.c
krishnasree08@krishnasree08: ~/practical8$ nano memory.c
krishnasree08@krishnasree08: ~/practical8$ gcc memory.c -o memory
krishnasree08@krishnasree08: ~/practical8$ ./memory
Using malloc(): 1 2 3 4 5
Using calloc(): 0 0 0 0 0
After realloc(): 1 2 3 4 5 6 7 8 9 10
Memory freed successfully
krishnasree08@krishnasree08: ~/practical8$ valgrind --version
valgrind-3.22.0
krishnasree08@krishnasree08: ~/practical8$ nano memory.c
krishnasree08@krishnasree08: ~/practical8$ gcc memory.c -o memory
krishnasree08@krishnasree08: ~/practical8$ valgrind --leak-check=full ./memory
==51478== Memcheck, a memory error detector
==51478== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==51478== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==51478== Command: ./memory
==51478==
Using malloc(): 1 2 3 4 5
Using calloc(): 0 0 0 0 0
After realloc(): 1 2 3 4 5 6 7 8 9 10
Memory freed successfully
==51478==
==51478== HEAP SUMMARY:
==51478==   in use at exit: 60 bytes in 2 blocks
==51478==   total heap usage: 4 allocs, 2 frees, 1,104 bytes allocated
==51478==
==51478== 20 bytes in 1 blocks are definitely lost in loss record 1 of 2
==51478==    at 0x484D953: calloc (in /usr/libexec/valgrind/vgpreload_memch
eck-amd64-linux.so)
==51478==    by 0x1092DD: main (in /home/vennela_07/practical8/memory)
==51478==
==51478== 40 bytes in 1 blocks are definitely lost in loss record 2 of 2
==51478==    at 0x484DB80: realloc (in /usr/libexec/valgrind/vgpreload_memc
heck-amd64-linux.so)
==51478==    by 0x10937B: main (in /home/vennela_07/practical8/memory)
==51478==
==51478== LEAK SUMMARY:
==51478==   definitely lost: 60 bytes in 2 blocks
==51478==   indirectly lost: 0 bytes in 0 blocks
==51478==   possibly lost: 0 bytes in 0 blocks
```

```
==51478==      possibly lost: 0 bytes in 0 blocks
==51478==      still reachable: 0 bytes in 0 blocks
==51478==      suppressed: 0 bytes in 0 blocks
==51478==
==51478== For lists of detected and suppressed errors, rerun with: -s
==51478== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
krishnasree08@krishnasree08: ~/practical8$ nano memory.c
krishnasree08@krishnasree08: ~/practical8$ gcc memory.c -o memory
krishnasree08@krishnasree08: ~/practical8$ valgrind --leak-check=full ./memory
==51660== Memcheck, a memory error detector
==51660== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==51660== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==51660== Command: ./memory
==51660==
Using malloc(): 1 2 3 4 5
Using calloc(): 0 0 0 0 0
After realloc(): 1 2 3 4 5 6 7 8 9 10
Memory freed successfully
==51660==
==51660== HEAP SUMMARY:
==51660==      in use at exit: 0 bytes in 0 blocks
==51660==    total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==51660==
==51660== All heap blocks were freed -- no leaks are possible
==51660==
==51660== For lists of detected and suppressed errors, rerun with: -s
==51660== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *a, *b, *temp;
    int i;

    // malloc()
    a = (int *)malloc(5 * sizeof(int));

    if (a == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++) {
        a[i] = i + 1;
    }

    printf("Memory allocated using malloc():\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    // calloc()
    b = (int *)calloc(5, sizeof(int));

    if (b == NULL) {
        printf("Memory allocation failed\n");
        free(a);
        return 1;
    }

    printf("Memory allocated using calloc():\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", b[i]);
    }
    printf("\n");
}
```

Task 2 : Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
vennela_07@VennelaPC: ~/pr  X + v
krishnasree08@krishnasree08: ~$ cd practical8
krishnasree08@krishnasree08: ~/practical8$ nano cow.c
krishnasree08@krishnasree08: ~/practical8$ gcc cow.c -o cow
krishnasree08@krishnasree08: ~/practical8$ ./cow
Parent process PID: 54263

Memory usage of Parent BEFORE fork():
VmSize:    53892 kB
VmRSS:     52936 kB

Child process PID: 54270

Memory usage of Child BEFORE modification:
VmSize:    53892 kB
VmRSS:     52264 kB

Child is modifying the shared memory...

Memory usage of Child AFTER modification:
VmSize:    53892 kB
VmRSS:     52264 kB

Memory usage of Parent AFTER child modification:
VmSize:    53892 kB
VmRSS:     52936 kB
```

```
GNU nano 7.2                                COW.C *
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (50 * 1024 * 1024)

void show_memory(const char *process)
{
    char command[100];

    printf("\nMemory usage of %s:\n", process);

    sprintf(command, "grep -E 'VmSize|VmRSS' /proc/%d/status", getpid());

    system(command);
}

int main()
{
    char *data;

    // Allocate 50 MB memory
    data = (char *)malloc(SIZE);

    if (data == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }
}
```

```

GNU nano 7.2                                cow.c *
    printf("Memory allocation failed\n");
    return 1;
}

| // Write to every page so that memory is actually allocated
  for (int i = 0; i < SIZE; i += 4096)
  {
      data[i] = 'A';
  }

  printf("Parent process PID: %d\n", getpid());

  show_memory("Parent BEFORE fork()");

  pid_t pid = fork();

  if (pid < 0)
  {
      printf("fork() failed\n");
      free(data);
      return 1;
  }

  if (pid == 0)
  {
      // Child process
      printf("\nChild process PID: %d\n", getpid());

      show_memory("Child BEFORE modification");

      printf("\nChild is modifying the shared memory...\n");

      // Modify the memory
      for (int i = 0; i < SIZE; i += 4096)
      {
          data[i] = 'B';
      }

      show_memory("Child AFTER modification");

      free(data);
      exit(0);
  }
  else
  {
      // Parent process
      wait(NULL);

      show_memory("Parent AFTER child modification");

      free(data);
  }

  return 0;
}

```